

Sesión de clase 06 · Laboratorio: recomendador semántico de productos

Contexto: la tienda recibe quejas porque sus recomendaciones «parecen genéricas» (comentario real en `comentarios.csv`, calificación ★★☆☆☆). Construimos un recomendador que usa el **significado** de los productos y el **historial** de cada cliente, en vez de reglas fijas.

Datos:

- `data/productos.csv` — 90 productos, 9 categorías × 10.
- `data/resenas_entrega.csv` — 110 reseñas de entrega sobre 54 productos, de 63 clientes.

Etapas: A. Indexación · B. Búsqueda semántica · C. Perfil de cliente · D. Recomendar · E. Análisis.

Cierra con un reto opcional: agrupar `comentarios.csv` por tema con k-means.

```
import numpy as np
import pandas as pd
import chromadb
from chromadb.utils import embedding_functions

pd.set_option("display.max_colwidth", 100)

MODELO = "paraphrase-multilingual-MiniLM-L12-v2"
ef = embedding_functions.SentenceTransformerEmbeddingFunction(model_name=MODELO)
```

Loading weights: 100%  199/199 [00:00<00:00, 8595.42it/s]

Eta A — Indexación

Cargamos `productos.csv`, armamos el texto a vectorizar (nombre + categoría + descripción) e insertamos los 90 productos en una colección persistente de Chroma, con `categoria`, `subcategoria`, `precio` y `calificacion_promedio` como metadatos para poder filtrar después.

Usamos `PersistentClient` (guarda la base en disco, en `../bdv`) en vez de `Client()`: así la colección sobrevive entre ejecuciones del notebook y no hay que re-vectorizar los 90 productos cada vez.

```
productos = pd.read_csv("../data/productos.csv")
print(productos.shape)
productos[["id", "nombre", "categoria", "subcategoria", "precio",
"calificacion_promedio"]].head()
```

(90, 11)

	id	nombre	categoria	subcategoria	precio	calificacion_promedio
0	1	Apple iPhone 15 Pro Max 256GB	Smartphones	Gama Alta	5599	3.5
1	2	Apple iPhone 15 128GB	Smartphones	Gama Alta	3799	3.3
2	3	Samsung Galaxy S24 Ultra 512GB	Smartphones	Gama Alta	5299	3.3
3	4	Samsung Galaxy A55 128GB	Smartphones	Gama Media	1399	3.0
4	5	Xiaomi Redmi Note 13 Pro 256GB	Smartphones	Gama Media	999	2.8

```

cliente_bdv = chromadb.PersistentClient(path="../bdv")

col_productos = cliente_bdv.get_or_create_collection(
    name="productos",
    embedding_function=ef,
    metadata={"hnsw:space": "cosine"},
)

texto_producto = productos["nombre"] + ". " + productos["categoria"] + ". " +
productos["descripcion"]

if col_productos.count() == 0:
    col_productos.add(
        ids=productos["id"].astype(str).tolist(),
        documents=texto_producto.tolist(),
        metadatas=productos[["categoria", "subcategoria", "precio",
"calificacion_promedio"]].to_dict("records"),
    )

col_productos.count()

```

90

Etapas B — Búsqueda semántica

Tres consultas en lenguaje natural: una libre, una filtrada por precio (`where`) y otra filtrada por categoría.

```

def buscar_productos(consulta, n_results=5, where=None):
    res = col_productos.query(query_texts=[consulta], n_results=n_results,
where=where)
    filas = []
    for doc, dist, meta in zip(res["documents"][0], res["distances"][0],
res["metadatas"][0]):
        filas.append({
            "similitud": round(1 - dist, 3),
            "categoria": meta.get("categoria"),
            "precio": meta.get("precio"),
            "producto": doc.split(".")[0],
        })
    return pd.DataFrame(filas)

buscar_productos("algo para escuchar musica mientras hago deporte")

```

	similitud	categoria	precio	producto
0	0.393	Audio	189	JBL Tune 510BT
1	0.362	Televisores y Video	3299	Sony Bravia X75L 65 pulgadas
2	0.359	Gaming y Consolas	349	HyperX Cloud II
3	0.353	Gaming y Consolas	799	SteelSeries Arctis Nova 7
4	0.331	Audio	999	Apple AirPods Pro 2da Generacion

```

buscar_productos(
    "regalo para quien empieza en fotografia",
    where={"precio": {"$lt": 2000}},
)

```

	similitud	categoria	precio	producto
0	0.433	Fotografia y Video	349	Fujifilm Instax Mini 12
1	0.379	Fotografia y Video	1899	Canon EOS Rebel T7
2	0.322	Fotografia y Video	1699	DJI Osmo Action 4
3	0.271	Accesorios y Componentes	279	Logitech C920 Webcam HD Pro
4	0.267	Electrodomesticos Inteligentes	599	Philips Hue White and Color Kit de Inicio

```

buscar_productos(
    "equipo para jugar en casa con amigos",
    where={"categoria": "Gaming y Consolas"},
)

```

	similitud	categoria	precio	producto
0	0.256	Gaming y Consolas	399	Razer Kishi V2
1	0.243	Gaming y Consolas	189	Logitech G502 Hero
2	0.236	Gaming y Consolas	349	HyperX Cloud II
3	0.223	Gaming y Consolas	2699	Sony PlayStation 5 Slim
4	0.209	Gaming y Consolas	799	SteelSeries Arctis Nova 7

Etapas C — Perfil de cliente

$\text{perfil} = \sum w_i \cdot v_i / \sum w_i$, donde v_i es el embedding de cada producto reseñado por el cliente y w_i un peso según la calificación de su reseña. Recuperamos los embeddings ya guardados en Chroma con `col.get(..., include=["embeddings"])`, en vez de volver a vectorizar los productos.

Se calcula el perfil de los clientes 1, 85 y 22 (los que más reseñas tienen: 10, 5 y 4 respectivamente).

```

resenas = pd.read_csv("../data/resenas_entrega.csv")

resenas["cliente_id"].value_counts().head(5)

```

```

cliente_id
1         10
85         5
33         4
22         4
116        3
Name: count, dtype: int64

```

```

def perfil_cliente(cliente_id, resenas, col_productos, peso=lambda calificacion:
calificacion):
    hist = resenas[resenas["cliente_id"] == cliente_id]
    ids_productos = hist["producto_id"].astype(str).unique().tolist()

    embeddings = col_productos.get(ids=ids_productos, include=["embeddings"])
    vectores = {pid: emb for pid, emb in zip(embeddings["ids"],
embeddings["embeddings"])}

    pesos_totales = 0.0
    acumulado = np.zeros(len(next(iter(vectores.values()))))
    for _, fila in hist.iterrows():
        pid = str(fila["producto_id"])
        w = peso(fila["calificacion"])
        acumulado += w * np.array(vectores[pid])
        pesos_totales += w

    return acumulado / pesos_totales, ids_productos

clientes_objetivo = [1, 85, 22]
perfiles = {}
for cid in clientes_objetivo:
    vector, productos_comprados = perfil_cliente(cid, resenas, col_productos)
    perfiles[cid] = {"vector": vector, "comprados": productos_comprados}
    print(f"Cliente {cid}: perfil calculado a partir de {len(productos_comprados)}
producto(s) reseñado(s) -> {productos_comprados}")

```

Cliente 1: perfil calculado a partir de 3 producto(s) reseñado(s) -> ['63', '3', '33']

Cliente 85: perfil calculado a partir de 3 producto(s) reseñado(s) -> ['43', '73', '13']

Cliente 22: perfil calculado a partir de 4 producto(s) reseñado(s) -> ['44', '53', '83', '23']

Etapas D — Recomendar

Consultamos la colección con `query_embeddings` (el perfil ya es un vector, no hace falta re-vectorizarlo), pedimos más resultados de los necesarios y descartamos los productos que el cliente ya compró.

```

def recomendar(cliente_id, perfiles, col_productos, productos_df, top_n=5):
    perfil = perfiles[cliente_id]["vector"]
    comprados = set(perfiles[cliente_id]["comprados"])

    res = col_productos.query(
        query_embeddings=[perfil.tolist()],
        n_results=top_n + len(comprados),
    )

    filas = []

```

```

for pid, dist in zip(res["ids"][0], res["distances"][0]):
    if pid in comprados:
        continue
    nombre = productos_df.loc[productos_df["id"].astype(str) == pid,
"nombre"].values[0]
    categoria = productos_df.loc[productos_df["id"].astype(str) == pid,
"categoria"].values[0]
    filas.append({"producto_id": pid, "producto": nombre, "categoria": categoria,
"similitud": round(1 - dist, 3)})
    if len(filas) == top_n:
        break

return pd.DataFrame(filas)

for cid in clientes_objetivo:
    print(f"\nTop-5 recomendaciones para el cliente {cid}:")
    display(recomendar(cid, perfiles, col_productos, productos))

```

Top-5 recomendaciones para el cliente 1:

	producto_id	producto	categoria	similitud
0	31	LG OLED55C3 55 pulgadas	Televisores y Video	0.745
1	4	Samsung Galaxy A55 128GB	Smartphones	0.741
2	5	Xiaomi Redmi Note 13 Pro 256GB	Smartphones	0.709
3	9	Samsung Galaxy A15 128GB	Smartphones	0.699
4	39	Sony Bravia X75L 65 pulgadas	Televisores y Video	0.675

Top-5 recomendaciones para el cliente 85:

	producto_id	producto	categoria	similitud
0	6	Xiaomi Poco X6 Pro 512GB	Smartphones	0.642
1	37	LG 50 pulgadas UHD UR8000	Televisores y Video	0.635
2	19	HP Omen 16 RTX 4060	Laptops y Computadoras	0.633
3	87	Corsair Vengeance 16GB DDR4 3200MHz	Accesorios y Componentes	0.621
4	12	Apple MacBook Pro 14 pulgadas M3 512GB	Laptops y Computadoras	0.618

Top-5 recomendaciones para el cliente 22:

	producto_id	producto	categoria	similitud
0	58	Amazfit GTS 4 Mini	Wearables y Fitness	0.749
1	60	Huawei Watch GT 4	Wearables y Fitness	0.725
2	52	Apple Watch Ultra 2	Wearables y Fitness	0.682
3	59	Garmin Venu 3	Wearables y Fitness	0.672
4	4	Samsung Galaxy A55 128GB	Smartphones	0.666

Etapas E — Análisis

1. Arranque en frío. Un cliente con una sola reseña tiene un perfil igual al embedding de un único producto: las recomendaciones quedan muy pegadas a ese producto y no reflejan un "gusto"

general. Con cero reseñas no hay perfil que calcular - ahí se necesita otra estrategia (productos populares, o pedir preferencias explícitas).

2. Peso de una reseña de 1★. Hasta ahora usamos `peso = calificacion`, así que una reseña de 1★ sigue sumando en la misma dirección que una de 5★, solo que con menos peso - el perfil nunca se aleja de un producto que al cliente **no le gustó**. Probemos la alternativa sugerida en clase: `peso = calificacion - 3`, que vuelve negativas las reseñas de 1★ y 2★ (el perfil se aleja de esos productos) y positivas las de 4★ y 5★.

```
def perfil_cliente_centrado(cliente_id, resenas, col_productos):
    return perfil_cliente(cliente_id, resenas, col_productos, peso=lambda
calificacion: calificacion - 3)

for cid in clientes_objetivo:
    hist = resenas[resenas["cliente_id"] == cid][["producto_id", "calificacion"]]
    if (hist["calificacion"] <= 2).any() and hist["producto_id"].nunique() > 1:
        vector_centrado, comprados = perfil_cliente_centrado(cid, resenas,
col_productos)
        vector_original = perfiles[cid]["vector"]
        similitud_entre_perfiles = float(
            np.dot(vector_original, vector_centrado)
            / (np.linalg.norm(vector_original) * np.linalg.norm(vector_centrado))
        )
        print(f"Cliente {cid}: coseno entre el perfil con peso=calificacion y
peso=calificacion-3: ")
            f"{similitud_entre_perfiles:.3f}")
        perfiles_centrado = {cid: {"vector": vector_centrado, "comprados":
comprados}}
        print("Recomendaciones con el perfil centrado (peso = calificacion - 3):")
        display(recomendar(cid, perfiles_centrado, col_productos, productos))
        break
    else:
        print("Ninguno de los clientes objetivo tiene una reseña de 1-2 estrellas junto
con otras; ")
            "prueben con otro cliente_id de resenas_entrega.csv para ver el efecto.")
```

Cliente 22: coseno entre el perfil con peso=calificacion y peso=calificacion-3: 0.893
Recomendaciones con el perfil centrado (peso = calificacion - 3):

	producto_id	producto	categoria	similitud
0	60	Huawei Watch GT 4	Wearables y Fitness	0.666
1	58	Amazfit GTS 4 Mini	Wearables y Fitness	0.656
2	52	Apple Watch Ultra 2	Wearables y Fitness	0.629
3	27	Samsung Galaxy Buds2 Pro	Audio	0.616
4	51	Apple Watch Series 9 GPS 41mm	Wearables y Fitness	0.615

3. De Chroma embebido a un servidor con miles de usuarios. Con

`Client()` / `PersistentClient()` todo corre en el mismo proceso Python: no hay concurrencia real, no hay autenticación ni control de acceso, y no se puede escalar horizontalmente. Con miles de usuarios simultáneos haría falta el modo servidor de Chroma (o Milvus/Qdrant/un servicio gestionado), separar el proceso de ingesta del de consulta, y decidir el índice ANN (HNSW por defecto) y sus parámetros de recall/latencia según el tráfico esperado.

Reto opcional: agrupar comentarios.csv por tema

Indexamos `comentarios.csv` en una segunda colección y agrupamos sus embeddings con k-means (sin usar la BDV para el clustering en sí, solo para guardar y reutilizar los vectores ya calculados).

```
from sklearn.cluster import KMeans

comentarios = pd.read_csv("../data/comentarios.csv")

col_comentarios = cliente_bdv.get_or_create_collection(
    name="comentarios",
    embedding_function=ef,
    metadata={"hnsw:space": "cosine"},
)

if col_comentarios.count() == 0:
    col_comentarios.add(
        ids=comentarios["id"].astype(str).tolist(),
        documents=comentarios["texto"].tolist(),
    )

datos = col_comentarios.get(
    ids=comentarios["id"].astype(str).tolist(),
    include=["embeddings", "documents"],
)
X = np.array(datos["embeddings"])

N_CLUSTERS = 6
kmeans = KMeans(n_clusters=N_CLUSTERS, random_state=42, n_init="auto").fit(X)

for cluster_id in range(N_CLUSTERS):
    centro = kmeans.cluster_centers_[cluster_id]
    distancias = np.linalg.norm(X - centro, axis=1)
    idx_representativo = int(np.argmin(distancias))
    tamano = int((kmeans.labels_ == cluster_id).sum())
    print(f"Cluster {cluster_id} ({tamano} comentarios) – ejemplo más cercano al centroide:")
    print(" ", datos["documents"][idx_representativo])
    print()
```

Cluster 0 (18 comentarios) – ejemplo más cercano al centroide:

El sitio funciona bien en general pero el checkout como invitado a veces pide iniciar sesion de todas formas. Es un detalle menor pero confunde un poco.

Cluster 1 (24 comentarios) – ejemplo más cercano al centroide:

Un vendedor de soporte me dio informacion incorrecta sobre un plazo de entrega y tuve que reprogramar mis planes.

Cluster 2 (13 comentarios) – ejemplo más cercano al centroide:

Me encanto poder pagar en cuotas sin intereses, facilito mucho la compra.

Cluster 3 (46 comentarios) – ejemplo más cercano al centroide:

La seccion de resenas de otros clientes me ayudo bastante a decidir que comprar.

Cluster 4 (27 comentarios) – ejemplo más cercano al centroide:

El seguimiento del pedido no siempre se actualiza en tiempo real, a veces se queda igual por dos dias. Al final el paquete llego bien pero la informacion no era precisa.

Cluster 5 (7 comentarios) – ejemplo más cercano al centroide:

La comparacion de especificaciones tecnicas me ayudo muchisimo a elegir entre dos laptops similares. Es justo el tipo de informacion que uno busca antes de invertir en tecnologia.

Cada cluster agrupa comentarios que hablan de temas parecidos (envíos, pagos, soporte, calidad del sitio, etc.) sin haber definido esas categorías a mano — a diferencia del Ejercicio 2, donde las etiquetas se definieron antes de clasificar.